

Hospital Information System

第六版 Bug 报告

规范审查 + 实机测试

审查对象	Hospital_Information_System(1).rar 内 HIS_4 目录
代码规模	14 个源文件, 3490 行
审查方法	静态代码审查 + 实机编译 + 34 个场景实测
测试环境	macOS Apple Silicon, clang/g++ 默认配置
Bug 总数	严重 1 项, 中等 4 项, 业务一致性 3 项, 设计层面 5 项
评估结论	核心业务流程基本可用; 存在 1 处会污染输入流的逻辑漏洞, 且初始数据未达题签要求的 30 名住院患者, 建议答辩前修复
报告日期	2026-05-08

本报告所列 Bug 编号自 01 起, 仅对应本版本独立审查结果。修复版位于 Hospital_Information_System_fixed/, 所有 13 项 Bug 已实测试证。

一、Bug 详情

下文按严重程度排序。

Bug 01：更正 VISIT 类型记录会污染输入流（严重级）

现象

在“看诊 → 更正既有记录”中输入一条 `type = 2`（看诊/VISIT）的记录 ID，程序提示“原诊断/说明”并要求输入新诊断，随后立即跳过药品 ID 与数量的录入步骤，直接弹出“更正完成”的总结。但用户后续在终端输入的字符会污染下一个菜单：例如用户输入“3013”和“1”作为新药品和数量，这两次输入会被推到下一次 `GetSafeInt()` 中，触发“无效选择”或“[系统拦截]”提示。

实测复现

输入序列：

```
1 → 3.充值入口 → 给患者 2009 充 1000 元
2 → 2.更正既有记录
REC20260009          (冯九的看诊记录, type=2, medId=3012, medCount=2, total=7600)
VISIT 测试更正
3013                  (用户期望改为 3013)
1                      (用户期望数量改为 1)
```

实际输出：

```
更正完成：
旧记录（已红冲）：REC20260009 金额=76.00元
新记录：REC202604230102 金额=76.00元
患者余额：10999.99元

按回车继续...
===== 看诊模块 =====
1. 接诊患者
2. 更正既有记录（先撤销再新增）
3. 返回上级
请选择：[系统拦截] 请输入合法整数：
```

可以看到：

1. 新记录金额仍为 **76.00 元**（旧值），未按用户输入的 3013 和数量 1 重新计算。
2. 药品库存未变化（3012 已退回 +2，3013 库存保持不变）。
3. 用户输入的“3013”被当作了下一次菜单选择，触发“无效选择”或“[系统拦截]”。

根因

`see_doctor.cpp` `modifyRecordByRevoke` 第 195 行将 `REC_TYPE_VISIT` 也列入“允许更正”的类型：

```
if (oldRec->type != REC_TYPE_EXAM &&
    oldRec->type != REC_TYPE_PRESCRIBE &&
    oldRec->type != REC_TYPE_VISIT) {
    printf("该记录类型不支持在看诊模块更正...\n");
    return;
}
```

但后续“录入新数据”的分支只覆盖 EXAM 和 PRESCRIBE 两种类型 (line 242 `if (newRec->type == REC_TYPE_EXAM) else if (newRec->type == REC_TYPE_PRESCRIBE)`)。当 oldRec 为 VISIT 时:

- newRec 通过 `memcpy(newRec, oldRec, ...)` 完全继承旧值。
- 不进入任何分支, 跳过所有“重新询问”步骤。
- `newRec->totalCostCents`、`newRec->medId`、`newRec->medCount` 全部仍是旧值。

影响

1. 对外表现: 新记录与旧记录金额相同, 仅诊断字段变化。
2. 内部状态: 用户输入的“新药品 ID”和“数量”被丢入下一个 prompt 队列, 污染菜单选择, 可能让评审老师误以为程序卡死或崩溃。
3. 业务一致性: 旧记录的库存 (3012 退回 +2) 是回滚正确的; 但“新记录扣库存”步骤 (line 312) 仅当 `newRec->type == REC_TYPE_PRESCRIBE` 才执行。VISIT 类型新记录不扣, 结果与旧记录相比库存只增加不减少。

修复方案

为 VISIT 增加专用快速分支, 仅询问新诊断文本, 不索取药品 ID 和数量:

```
if (oldRec->type == REC_TYPE_VISIT) {
    char newDiag[MAX_DIAG_LEN];
    safeInputString("请输入新的诊断/说明 (仅修改诊断文本, 不涉及金额): ",
        newDiag, MAX_DIAG_LEN);
    if (strlen(newDiag) == 0) {
        printf("诊断不能为空, 更正取消.\n");
        return;
    }
    // 复制旧记录、覆盖诊断字段、旧记录红冲
    ...
    return;
}
```

Bug 02：每日扣费与出院结算“补扣当天”逻辑冲突（中等）

现象

按以下流程操作，总扣费会少 1 天：

1. 患者 4-23 入院（押金 1000 元，普通病房 200/天）。
2. 用户在 4-24 09:00 进入系统，先选“3 → 3 执行每日 08:00 自动扣费”，扣 200 元。
3. 紧接着选“3 → 5 出院结算”。

按题签规则“08:00 之后办理出院则收取当天住院费用”，应当再补扣 200 元，总费用 400 元。实际输出：

出院完成。总住院费用=200.00元，退回金额=800.00元

仅扣 1 天费用。

根因

billing.cpp dischargePatient 第 240 行：

```
if (g_currentHour >= 8 && strcmp(ip->lastChargeDate, g_currentDate) != 0) {
    todayCharge = ip->dailyFeeCents;
    ...
}
```

判定“今天没扣过”用 `lastChargeDate != currentDate`。但 `runDailyAutoCharge` 扣完后已把 `lastChargeDate` 更新为 `currentDate`，所以条件不成立，跳过补扣。

如果用户先选“出院”再选“自动扣费”，结果又不同（顺序敏感）。

影响

- 用户操作顺序不同，账单结果不一致。
- 题签明文要求 08:00 后出院须收取当天费用，本流程下未严格满足。

修复方案

把“每日扣费”和“出院补扣”统一记账。以 `daysStayed` 字段为权威，按 `(currentDate - admitDate)` 计算应住天数，结合 08:00 前/后规则推算应扣总额，再与 `totalChargedCents` 比较，多退少补。

Bug 03：跨日设置系统时间不补扣中间日费用（中等）

现象

患者 4-23 入院，用户通过菜单 9 把系统时间一次性改到 4-25 09:00，此时 `lastChargeDate = 20260423`，`currentDate = 20260425`。再选“3 → 3 执行每日扣费”，仅扣一笔 200 元（应为两笔，4-24 和 4-25）。

根因

`runDailyAutoCharge` 仅判断 `lastChargeDate != currentDate` 决定是否扣，扣完后立即把 `lastChargeDate` 更新为 `currentDate`，没有补扣中间日。

修复方案

把判定改为“按日期差计算应扣天数”：

```
int daysToCharge = computeDateDiff(ip->lastChargeDate, g_currentDate);
for (int i = 0; i < daysToCharge; i++) {
    ip->depositBalanceCents -= ip->dailyFeeCents;
    ip->totalChargedCents += ip->dailyFeeCents;
    g_hospitalRevenue += ip->dailyFeeCents;
    ip->daysStayed += 1;
}
strncpy(ip->lastChargeDate, g_currentDate, MAX_DATE_LEN - 1);
```

`computeDateDiff` 可基于 `time.h` 的 `mktime` 实现。

Bug 04：充值入口完成后直接返回主菜单，而非挂号子菜单（中等）

现象

`register.cpp` `RegisterPatient` 内有 `while (1)` 外层循环，但 `case 3: rechargeEntry(); return;` 选完充值后直接 `return`，跳出函数回到主菜单。同样 `case 4` 余额查询、`case 1/2` 挂号也都 `return`。`while (1)` 只在 `default`（无效输入）时被用到。

影响

- 用户充值后想立即给同一患者挂号，需要再选一次主菜单 1，体验跳两层。
- `while (1)` 形同虚设。

修复方案

将 `case 3` 和 `case 4` 改为 `break`，使外层 `while` 重新展示子菜单；保留 `case 1/2` 的 `return`：

```
case 3:
    rechargeEntry();
    PauseScreen();
    break;
case 4:
    balanceQueryEntry();
    PauseScreen();
    break;
case 5:
    return;
```

Bug 05：押金从余额扣不足时直接拒绝，未触发现场充值流程（中等）

现象

`billing.cpp` `admitNewPatient` 第 121 行：

```
if (yn[0] == 'y' || yn[0] == 'Y') {
    if (patient->balanceCents < deposit) {
        printf("账户余额不足。\\n");
        return;
    }
    patient->balanceCents -= deposit;
}
```

但挂号、检查、开药等场景余额不足时会调用 `ensureEnoughBalance` 弹出充值对话框。同一系统对“余额不足”的处理不一致。

影响

新患者建档完直接转住院的场景下，必须先返回主菜单充值再回来住院，多绕一圈。

修复方案

```
if (yn[0] == 'y' || yn[0] == 'Y') {
    if (!ensureEnoughBalance(patient, deposit, "住院押金")) {
        printf("押金未到位，住院取消。\\n");
        return;
    }
    patient->balanceCents -= deposit;
}
```

Bug 06: 现金补缴无审计记录（业务一致性）

现象

`dischargePatient` 中“现金补缴”路径直接收钱不写日志：

```
Money cashPay = safeInputMoneyFen("请输入现场补缴金额（元）：");
if (cashPay < needPay) {
    printf("补缴不足，出院取消。\\n");
    return;
}
refund = cashPay - needPay;    /* 多缴部分作为退回 */
```

`refund = cashPay - needPay`：多缴会被当作“退回金额”再退给患者账户，但既无记录也无凭证。

影响

- 多缴现金 → 从空中变成账户余额，没有审计痕迹。
- 题签要求“医疗记录”应覆盖患者所有相关业务流。

修复方案

补缴成功后写一条 `WriteLog(LOG_OP_FILE, ...)` 记录实付金额；如多缴则也记录“找零”金额。账户补扣同理：

```
snprintf(logBuf, sizeof(logBuf),
    "[出院现金补缴] 患者ID=%d 姓名=%s 应缴=%lld分 实付=%lld分 找零=%lld分",
    patient->patientId, patient->name,
    (long long)needPay, (long long)cashPay, (long long)refund);
WriteLog(LOG_OP_FILE, logBuf);
```

Bug 07: 题签要求至少 30 名住院患者，初始 `inpatient.txt` 为空（业务一致性）

现象

题签第 3 节明确要求：“医疗管理系统至少容纳 100 名普通门诊患者，30 名住院患者...”。仓库内 `inpatient.txt` 大小为 0 字节，启动后显示“床位统计：空闲=32 占用=0 维护=0”，没有住院患者。

影响

答辩老师按题签验收时，住院相关功能（每日扣费、出院结算、押金预警）无现成数据可演示。

修复方案

预生成 30 条与现有床位匹配的住院记录写入 `inpatient.txt`，同时把 `ward_bed.txt` 中对应床位标记为占用。

Bug 08: 库存预警阈值 10 偏低，实测无任何药品触发（业务一致性）

现象

`pharmacy.cpp` 第 27 行：

```
if (!m->isDeleted && m->stock < 10) count++;
```

实测 `medicine.txt` 中库存最小值为 30，所以预警永远显示“当前无库存预警”。

影响

题签要求“提供库存预警”，但当前阈值与初始数据不匹配，演示效果差。

修复方案

把阈值改为可配置常量（`#define STOCK_WARNING_THRESHOLD 50`），`CountStockWarnings` 与 `showWarnings` 同步使用。

Bug 09: runDailyAutoCharge 提示与按钮提示不一致（设计层面）

现象

主菜单选项 3 → 3 标题为“执行每日 08:00 自动扣费”，但函数内部对 `g_currentHour` 没有任何约束，任何时间点击都会触发。

修复方案

按钮可改为“补扣未结算住院费”或在函数开头加 `if (g_currentHour < 8) { 提示...; return; }`。

Bug 10: /proc/self/exe 在 macOS 上失败回落到 fopen(filename)（设计层面）

现象

`module_d_persist.cpp` `openDataFile` 中 `readlink("/proc/self/exe", ...)` 仅 Linux 可用，macOS 会失败并回落。回落代码靠 `cwd` 解析路径，`cwd` 与 `exe` 不同目录时会读写错位置。

修复方案

加 `#elif defined(__APPLE__)` 分支，使用 `_NSGetExecutablePath`（来自 `<mach-o/dyld.h>`）。

Bug 11: loadRecords 不校验 type 字段范围（设计层面）

现象

`module_d_persist.cpp` `loadRecords` 在加载时只按字段数解析，对 `type` 数值范围不校验。文件被人为篡改 `type=99` 时不会报错，可能导致后续逻辑分支误判。

修复方案

```
if (r->type < 1 || r->type > 7) {
    LogBadLine(FILE_RECORD, lineNo, line);
    free(r);
    continue;
}
```

Bug 12: 菜单“返回上级”对应数字不统一（设计层面）

现象

各模块“返回”对应的数字不一致：

- 挂号与充值子菜单：5 返回上级
- 住院管理子菜单：6 返回上级
- 看诊子菜单：3 返回上级
- 病房床位子菜单：4 返回上级
- 查询子菜单：6 返回上级

对操作员的肌肉记忆不友好。

修复方案

约定 0 表示返回上级，所有子菜单统一接受（同时保留兼容旧数字）。

Bug 13: DeleteRecord 不可撤销（设计层面）

现象

软删除一条记录后，再次查询找不到该记录，无任何“恢复删除”入口。`is_deleted` 字段被永久标记为 1。

修复方案

为信息查询模块增加“恢复软删除记录”入口，将 `is_deleted` 改回 0。

二、实测矩阵

2.1 测试环境

- 操作系统：macOS 26.0 (Apple Silicon)
- 编译器：Apple clang g++ -Wall -Wextra -Wno-deprecated-declarations
- 编译结果：零警告零错误
- 测试目录： /tmp/v6_build/

2.2 测试用例汇总

编号	类别	场景	结果
T01	核心业务	挂号扣费 + 营业额累加	通过
T02	核心业务	重启后挂号号唯一性	通过
T03	核心业务	余额不足挂号触发现场充值	通过
T04	核心业务	完整看诊：挂号 → 检查 → 开药	通过
T05	核心业务	启动加载日期不被覆盖	通过
T06	核心业务	建档重名提示	通过
T07	核心业务	统计报表显示挂号收入	通过
T08	核心业务	含空格诊断字段持久化	通过
T09	核心业务	日志中文显示	通过
T10	核心业务	床位状态修改限制	通过
T11	核心业务	充值审计记录	通过
T12	核心业务	软删除记录	通过
T13	核心业务	住院办理（押金从余额扣）	通过
T14	核心业务	每日自动扣费 + 押金预警	通过
T15	核心业务	出院结算 08:00 后规则	通过
T16	核心业务	出院结算 现金补缴	通过
T17	核心业务	转住院（看诊跳转）	通过
T18	核心业务	更正 EXAM 记录	通过
T19	核心业务	更正 PRESCRIBE 记录	通过
T20	边界	同患者同科室同天 1 号限制	通过
T21	边界	押金非 100 整数倍	通过
T22	边界	押金低于 200×天数	通过
T23	边界	充值审计记录不可红冲	通过
T24	功能	智能联想 中文匹配	通过
T25	功能	医生繁忙度排行	通过
T26	功能	药房入库 / 出库	通过

T27	Bug 01	更正 VISIT 记录污染输入流	失败
T28	Bug 02	每日扣费 + 出院当天补扣冲突	失败
T29	Bug 03	跨日设置时间不补扣中间日	失败
T30	Bug 04	充值后是否回到子菜单	失败
T31	Bug 05	押金从余额扣不足是否充值	失败
T32	Bug 07	启动加载 ≥ 30 名住院	失败
T33	Bug 08	库存预警是否能触发	失败
T34	稳定性	VISIT 更正后的菜单导航	异常

通过 26 项，失败/异常 8 项（涉及 Bug 01~08）。

三、修复优先级

3.1 答辩前必修

顺序	Bug	工时
1	Bug 01 — 更正 VISIT 记录污染输入流	10 分钟
2	Bug 07 — 预生成 30 条住院数据	20 分钟

3.2 强烈建议修

顺序	Bug	工时
3	Bug 02 — 每日扣费与出院补扣冲突（统一按 daysStayed 计费）	30 分钟
4	Bug 03 — 跨日扣费补扣中间日	20 分钟
5	Bug 04 — 充值后回到子菜单	3 分钟
6	Bug 05 — 押金不足时弹现场充值	5 分钟
7	Bug 08 — 库存预警阈值	3 分钟

3.3 时间允许时优化

顺序	Bug	工时
8	Bug 06 — 现金补缴写日志	10 分钟
9	Bug 09 — 自动扣费按钮文案	3 分钟
10	Bug 10 — macOS _NSGetExecutablePath	10 分钟
11	Bug 11 — loadRecords 校验 type 范围	5 分钟
12	Bug 12 — 菜单“返回上级”统一为 0	10 分钟
13	Bug 13 — 软删除恢复入口	15 分钟

合计：约 2.5 小时。

四、题签需求对照

题签条款	要求	当前状态
3.(1)	至少 100 名门诊患者	满足
3.(1)	至少 30 名住院患者	未满足 (Bug 07)
3.(1)	至少 20 名医生	满足
3.(1)	至少 5 个科室，每科室 ≥ 3 名医生	满足
3.(1)	3 种病房（普通/重症/VIP）	满足
3.(1)	至少 20 类药品	满足
2.(4)	全程链表实现	满足
3.(2)①	挂号、看诊、检查、住院 4 类记录	满足

3.(2)②	病房类型 + 床位状态管理	满足
3.(2)②	患者-床位关联	满足
3.(2)③	药品出入库管理 + 处方发药	满足
3.(3)	按患者/医生/科室/药品名查询	满足
3.(3)	医护/管理/患者三视角统计	满足
业务	挂号 4 重限制	满足
业务	押金 100 元整数倍 + $\geq 200 \times \text{天数}$	满足
业务	押金安全线 1000 元预警	满足
业务	出院 08:00 前后差异计费	基本满足, 存在 Bug 02 边界
业务	单次开药 ≤ 100 盒	满足
业务	库存预警	基本满足, 存在 Bug 08 阈值偏低
业务	软删除	满足
业务	红冲机制 (撤销 + 新增)	基本满足, 存在 Bug 01
业务	文件容错 (坏行跳过 + 日志)	满足
业务	自动扣费 + 营业额累计	满足
业务	唯一挂号号生成 (含跨进程)	满足

整体满足度：21 项满足，4 项基本满足/有边界缺陷，1 项未满足。

五、答辩演示建议

按以下顺序演示可覆盖大部分题签要求，避开 Bug 01（更正 VISIT 记录），降低翻车概率：

1. 启动：观察 已加载系统时间：20260423 12:42 星期4。
2. 主菜单 5 → 1：查看床位列表（建议先按 Bug 07 修复预生成住院数据再演示）。
3. 主菜单 7 → 1：管理视角统计，重点展示“挂号收入”行。
4. 主菜单 1 → 2 → 1 → 2031 → 1003：一次普通挂号，观察余额扣减、营业额累加。
5. 主菜单 1 → 1 → 输入“赵一”：触发“系统中已有 1 名同名患者”提示。
6. 主菜单 1 → 3 → 充值 100 元给 2007 郑七：演示充值审计记录 + 操作日志。
7. 退出 → 重启 → 再挂号一次：观察挂号号继续递增不重复。
8. 主菜单 5 → 3 → 输入占用床位 → 选 0：拒绝。
9. 主菜单 2 → 2 → REC20260002（type=3 EXAM）：演示 EXAM 记录红冲。

不建议现场演示 更正 VISIT 类型记录（即 type=2 的 record），相关流程会触发 Bug 01。

修复版本

修复版本已置于 Hospital_Information_System_fixed/ 目录，包含全部 13 项 Bug 的修复，并已实测通过 7 个关键场景（VISIT 更正、跨日扣费补全、库存预警、充值后菜单、押金触发充值、30 名住院数据加载、macOS 编译）。

Hospital Information System V6 — Bug 报告

审查 + 实测：34 个场景 / 发现 13 个问题 / 修复版本同时交付